

IF2211 STRATEGI ALGORITMA
LAPORAN TUGAS BESAR 2



Disusun oleh:

Kelompok 26 – PengenLibur

Made Branenda Jordhy (13524026)

Irvin Tandiarrang Sumual (13524030)

Valentino Daniel Kusumo (13524104)

Dikumpulkan pada:

25 April 2026

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026

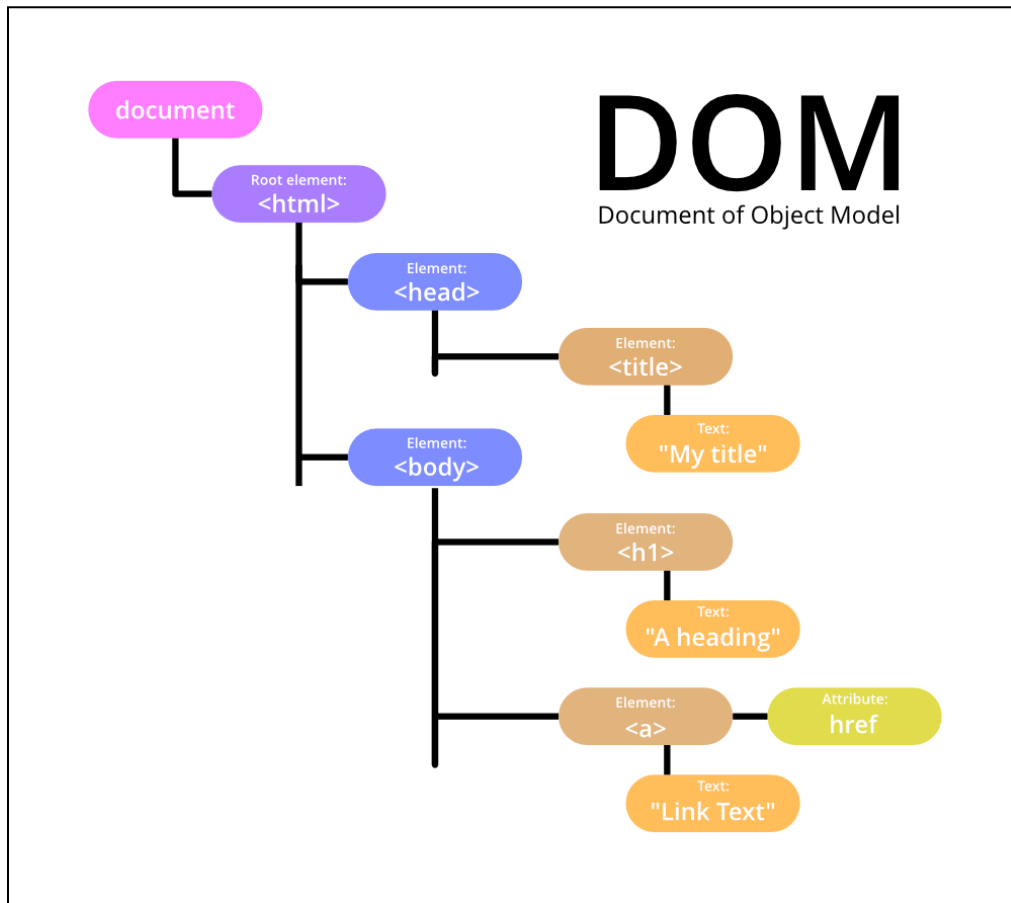
Daftar Isi

Daftar Isi.....	2
BAB I.....	4
1. Tags.....	5
2. CSS Selector.....	6
• Tag Selector.....	6
• Class Selector.....	7
• ID Selector.....	7
• Universal Selector.....	7
• Child Combinator (>).....	7
• Descendent Combinator (" ").....	7
• Adjacent Sibling Combinator (+).....	7
• General Sibling Combinator (~).....	7
3. HTML Parsing.....	7
BAB II.....	9
2.1 Tree.....	9
2.2 DFS.....	10
2.3 BFS.....	10
2.4 LCA.....	11
2.5 Scraping.....	12
2.6 Pengembangan Website.....	12
2.7 Bahasa Pemrograman.....	12
2.8 Multithreading.....	13
BAB III.....	14
3.1. BFS.....	14
3.2. DFS.....	15
3.3. LCA.....	16
BAB IV.....	18
4.1. Implementasi Algoritma BFS.....	18
4.1.1 Struktur Data.....	18
4.1.2 Fungsi dan Prosedur.....	18
4.2. Implementasi Algoritma DFS.....	22
4.2.1 Struktur Data.....	22
4.2.2 Fungsi dan Prosedur.....	22
4.3. Implementasi Algoritma Binary Lifting.....	26
4.3.1 Struktur Data.....	26
4.3.2 Fungsi dan Prosedur.....	27
4.4. Hasil Pengujian.....	30
1. Test Case 1.....	31
2. Test Case 2.....	32
3. Test Case 3.....	32
4. Test Case 4.....	33
5. Test Case 5.....	33
6. Test Case 6.....	34
7. Test Case 7.....	34

8. Test Case 8.....	35
9. Test Case 9.....	35
10. Test Case 10.....	36
11. Test Case 11.....	36
12. Test Case 12.....	37
13. Test Case 13.....	37
14. Test Case 14.....	38
BAB V.....	39
5.1 Kesimpulan.....	39
5.2 Saran.....	39
5.3 Refleksi.....	39
Lampiran.....	40
Pernyataan tidak melakukan kecurangan yang ditandatangani.....	40
Tautan repository GitHub.....	40
Daftar Pustaka.....	42

BAB I

Deskripsi Tugas



Gambar 1.1 Struktur Pohon DOM

<https://www.geeksforgeeks.org/java/what-is-document-object-in-java-dom/>

Document Object Model (DOM) merupakan representasi terstruktur dari dokumen HTML dalam bentuk tree yang memungkinkan setiap elemen pada halaman web diakses, dimodifikasi, dan dimanipulasi oleh program. Struktur ini merepresentasikan hubungan antar elemen HTML seperti parent, child, dan sibling sehingga memudahkan pengolahan dokumen secara terprogram.

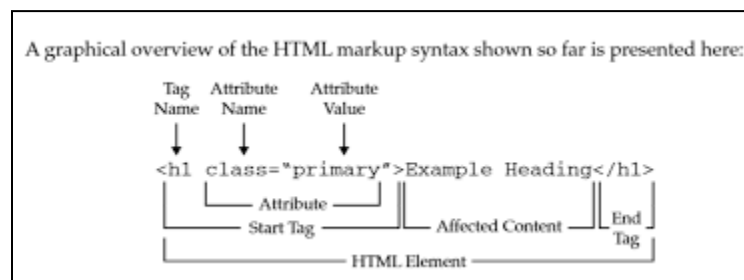
Struktur sebuah file HTML pada umumnya meliputi elemen root <html> yang berisi dua bagian utama yaitu <head> dan <body>. Bagian <head> berisi metadata seperti judul halaman, link stylesheet, dan script, sedangkan <body> berisi konten utama halaman seperti teks, gambar, tombol, dan elemen HTML lainnya yang ditampilkan kepada pengguna.

Sebuah website menampilkan sebuah DOM melalui sebuah file bernama Cascading Style Sheets (CSS) yang bertujuan mendeklarasikan visualisasi elemen-elemen pada pohon. Deklarasi visualisasi ditempelkan pada elemen-elemen yang berkaitan melalui CSS Selector.

Pada Tugas Besar 2 Strategi Algoritma ini, mahasiswa diminta untuk menelusuri dan mencari elemen pada struktur DOM berdasarkan CSS Selector tertentu dengan menggunakan algoritma penelusuran graf yaitu Breadth First Search (BFS) dan Depth First Search (DFS). Algoritma tersebut digunakan untuk menjelajahi struktur pohon DOM guna menemukan elemen yang sesuai dengan selector yang diberikan.

Komponen-komponen penting yang terdapat pada tugas besar ini antara lain:

1. Tags



Gambar 1.2 Struktur Syntax HTML

<https://tutorial.techaltum.com/htmlTags.html>

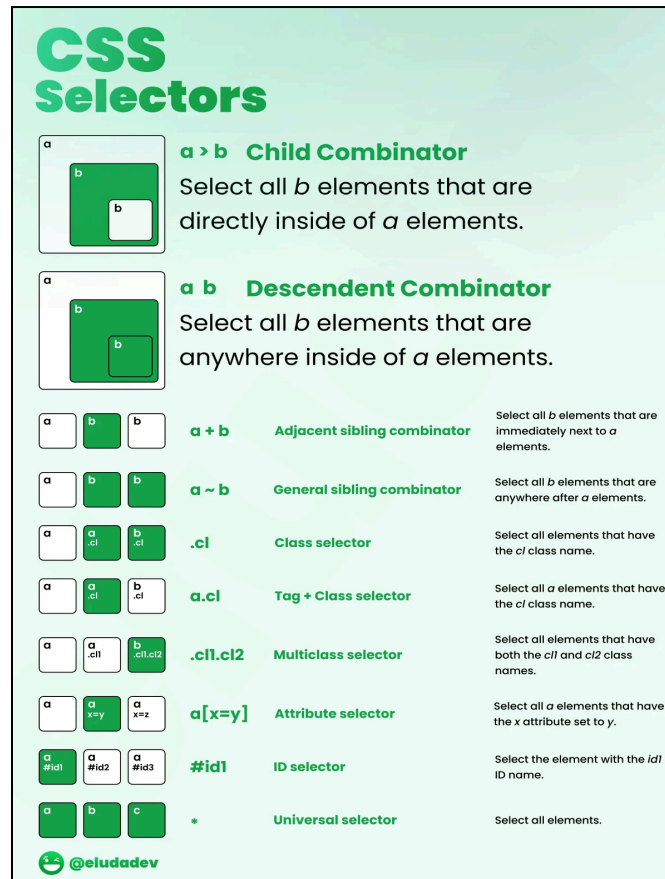
Tag pada HTML merupakan penanda (markup) yang digunakan untuk mendefinisikan struktur dan jenis elemen dalam sebuah dokumen HTML. Tag memberi tahu browser bagaimana suatu bagian konten harus ditafsirkan dan ditampilkan pada halaman web.

Secara umum, tag HTML ditulis menggunakan tanda kurung sudut (< >). Sebagian besar elemen HTML memiliki pasangan tag pembuka dan tag penutup. Tag pembuka menandai awal suatu elemen, sedangkan tag penutup menandai akhir elemen tersebut. Tag penutup ditulis dengan menambahkan tanda garis miring / sebelum nama tag. Terdapat beberapa jenis tag HTML yang self-closing.

Dalam struktur DOM, setiap tag HTML direpresentasikan sebagai node pada pohon DOM. Hubungan antar tag membentuk struktur hierarki seperti parent, child, dan

sibling . Struktur inilah yang memungkinkan dokumen HTML diproses sebagai sebuah pohon ketika dilakukan penelusuran menggunakan algoritma seperti BFS atau DFS.

2. CSS Selector



Gambar 1.3 Visualisasi CSS Selector

https://www.reddit.com/r/webdev/comments/ur6v5m/css_selectors_visually_explained/

CSS Selector adalah mekanisme pada CSS untuk memilih elemen HTML tertentu pada struktur DOM agar dapat diberikan aturan style. Dengan selector, kita dapat menentukan elemen mana yang akan dikenai aturan visual seperti warna, ukuran, atau tata letak. Beberapa selector dasar yang umum digunakan adalah sebagai berikut.

- Tag Selector

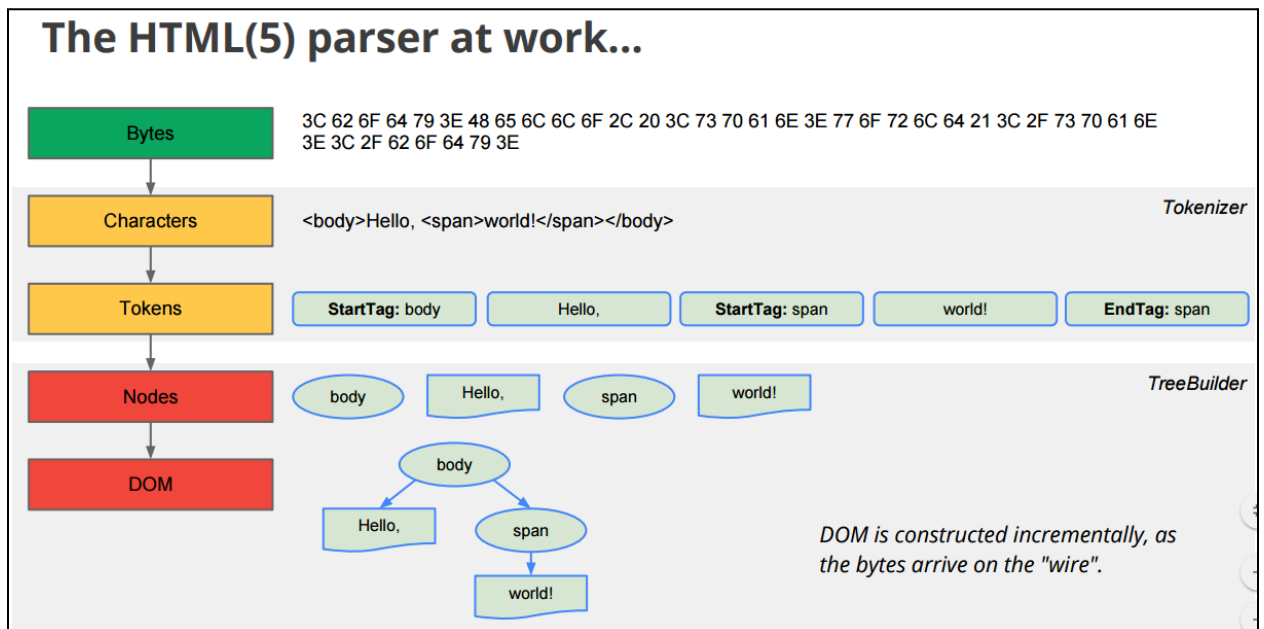
Memilih elemen berdasarkan nama tag HTML. Contoh: p memilih semua elemen $\langle p \rangle$.

- Class Selector
Memilih elemen yang memiliki atribut class tertentu, ditulis dengan tanda titik (.). Contoh: .box
- ID Selector
Memilih elemen dengan atribut id tertentu, ditulis dengan tanda pagar (#). Contoh: #header
- Universal Selector
Memilih semua elemen HTML. Contoh: *

Combinator merupakan cara untuk menggabungkan beberapa jenis selector dalam penentuan elemen yang akan terpengaruh, dan terdiri dari:

- Child Combinator (>)
- Descendent Combinator (" ")
- Adjacent Sibling Combinator (+)
- General Sibling Combinator (~)

3. HTML Parsing



Gambar 1.4 Proses Parsing Pohon HTML

<https://stackoverflow.com/questions/3150293/how-does-a-parser-for-example-html-work>

HTML Parsing adalah proses membaca dokumen HTML dan mengubahnya menjadi struktur data pohon (tree) yang merepresentasikan hubungan antar elemen HTML. Pada proses ini, parser memproses dokumen HTML secara berurutan, mengenali tag pembuka dan tag penutup, kemudian menyusunnya ke dalam struktur hierarki.

Setiap tag HTML direpresentasikan sebagai node pada pohon. Tag yang berada di dalam tag lain akan menjadi child node, sedangkan tag yang membungkusnya menjadi parent node. Node paling atas pada struktur ini disebut root, yang umumnya merupakan elemen `<html>`. Dari node root tersebut, elemen-elemen lain seperti `<head>` dan `<body>` akan menjadi cabang yang membentuk struktur pohon DOM.

Hasil dari proses parsing adalah DOM Tree, yaitu representasi pohon dari dokumen HTML yang memungkinkan elemen-elemen pada halaman web ditelusuri, diakses, dan dimanipulasi secara terstruktur. Struktur pohon ini kemudian dapat digunakan dalam proses penelusuran elemen, misalnya dengan menggunakan algoritma Breadth First Search (BFS) atau Depth First Search (DFS).

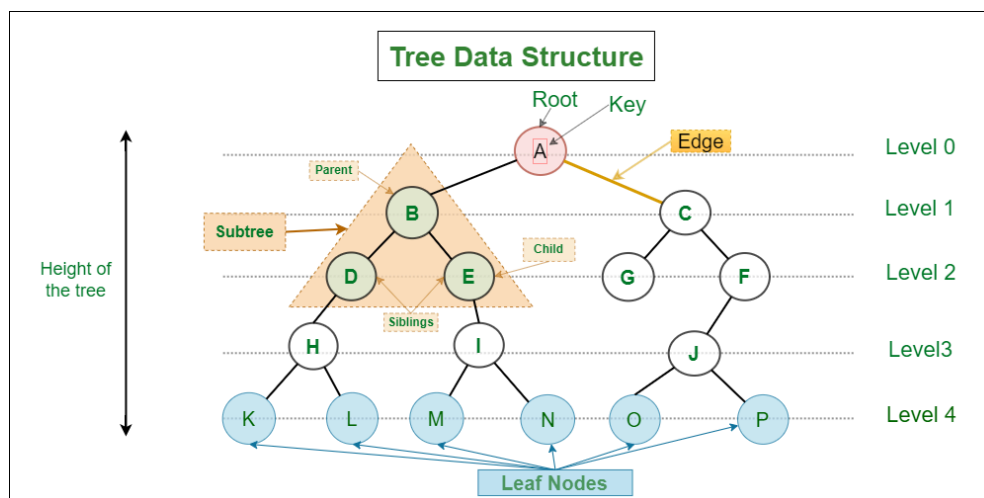
BAB II

Landasan Teori

2.1 Tree

Tree merupakan suatu struktur data nonlinear yang digunakan untuk merepresentasikan data dalam bentuk hierarki dengan hubungan parent-child. Tree terdiri dari kumpulan node yang biasanya berisikan informasi sesuai dengan domain persoalan yang akan dipecahkan. Node paling atas di dalam tree seringkali disebut sebagai root dan setiap node dapat memiliki nol atau lebih child. Terdapat juga sejumlah istilah penting di dalam tree.

- Parent Node: Node yang menjadi pendahulu langsung dari node lain
- Child Node: Node yang merupakan penerus langsung dari node lain
- Root Node: Node paling atas dalam tree yang tidak memiliki parent
- Leaf Node (External Node): Node yang tidak memiliki child
- Ancestor: Node yang berada pada jalur dari root ke suatu node (tidak termasuk node itu sendiri)
- Descendant: Node yang memiliki suatu node lain sebagai ancestor
- Sibling: Node-node yang memiliki parent yang sama
- Level of a Node: Jumlah edge dari root ke node tersebut (root berada di level 0)
- Internal Node: Node yang memiliki minimal satu child
- Neighbor of a Node: Parent atau child dari suatu node
- Subtree: Sebuah node beserta seluruh descendant-nya



Gambar 2.1 Contoh Representasi Tree Data Structure
(sumber : [Applications of tree data structure - GeeksforGeeks](#))

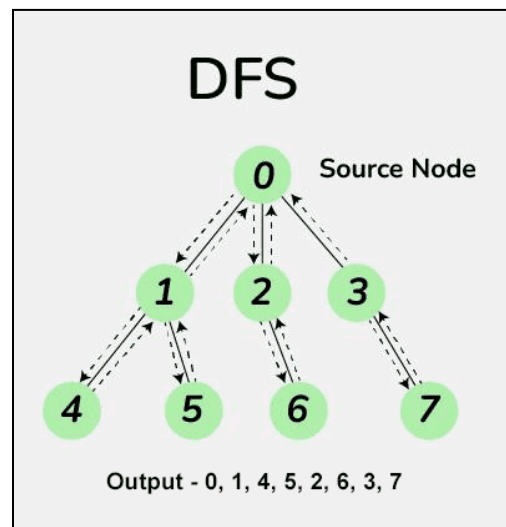
Struktur ini data tree digunakan untuk merepresentasikan data hierarkis seperti sistem file dan DOM pada HTML, serta membantu pengolahan dan pencarian

data secara efisien. Jenis tree bermacam-macam, seperti binary tree (maksimal dua anak), ternary tree (maksimal tiga anak), dan n-ary tree (jumlah anak bebas). Operasi dasar pada tree meliputi pembuatan, penyisipan, pencarian, dan traversal (DFS dan BFS). Selain itu, tree memiliki beberapa properti penting seperti jumlah edge ($N-1$ untuk N node), kedalaman node, tinggi tree, dan derajat node

2.2 DFS

Depth First Search (DFS) merupakan suatu algoritma yang digunakan untuk menelusuri suatu struktur data pohon (tree) ataupun graph. Algoritma DFS bekerja dengan cara menelusuri suatu *branch* pada *tree* sedalam mungkin sebelum melakukan *backtracking* untuk menelusuri *sibling* (node lainnya yang memiliki parent yang sama dengan node saat ini). Terdapat tiga tipe penelusuran (traversal) secara DFS, yaitu:

- Pre-order (secara rekursif mengunjungi root terlebih dahulu, diikuti dengan mengunjungi subpohon kiri, dan diakhiri dengan subpohon kanan)
- In-order (secara rekursif mengunjungi subpohon kiri terlebih dahulu, diikuti dengan root (akar), diakhiri dengan subpohon kanan)
- Post-order (secara rekursif mengunjungi subpohon kiri dan kanan terlebih dahulu lalu diakhiri dengan root).

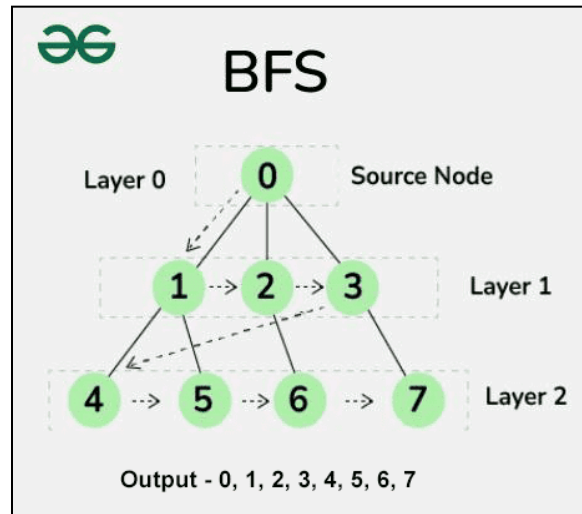


Gambar 2.2 Contoh DFS Traversal di suatu Tree
(sumber : [BFS vs DFS for Binary Tree - GeeksforGeeks](#))

2.3 BFS

Breadth First Search (BFS) merupakan suatu algoritma yang juga dapat digunakan untuk menelusuri suatu struktur data pohon (tree) ataupun graph. Algoritma BFS bekerja dengan cara menelusuri pohon secara *level by level*, mulai dari root, dilanjutkan dengan menelusuri setiap tetangganya sebelum lanjut ke level

berikutnya. Penerapan BFS umumnya menggunakan struktur data queue (First in First out), di mana node yang lebih dulu ditemukan akan diproses lebih dahulu. Karena penelusuran yang dilakukan merata di tiap levelnya, BFS sangat efektif untuk mencari jalur terpendek dalam suatu graph yang tidak berbobot.



Gambar 2.3 Contoh BFS Traversal di suatu Tree
(sumber : [BFS vs DFS for Binary Tree - GeeksforGeeks](#))

2.4 LCA

Lowest Common Ancestor merupakan salah satu permasalahan klasik terkait dengan tree-based algorithm. LCA merupakan konsep dalam tree yang merujuk pada node terdalam di suatu tree yang menjadi nenek moyang bersama dari dua node tertentu. Dengan kata lain, LCA merupakan node terdalam yang menjadi nenek moyang bersama dari dua node tertentu. Secara sederhana, LCA merupakan titik pertemuan paling dekat ke bawah dari dua node dalam suatu tree. Salah satu pendekatan sederhana untuk menentukan LCA ialah dengan menelusuri ke atas dari kedua node hingga menemukan ancestor yang sama. Akan tetapi, hal ini tidak akan efisien untuk tree yang besar.

Terdapat suatu algoritma yang dapat mengoptimasi proses pencarian LCA, yaitu dengan menggunakan teknik optimasi seperti *binary lifting*, yaitu metode preprocessing yang menyimpan ancestor setiap node dalam bentuk loncatan pangkat dua (misalnya saja 1, 2, 4, dan seterusnya) sehingga pergerakan ke atas tidak dilakukan satu per satu, melainkan dengan memanfaatkan lompatan cepat. Dengan optimasi ini, pencarian LCA dilakukan dengan dua tahap. Dimulai dengan menyamakan kedalaman kedua node dengan mengangkat node yang lebih dalam menggunakan lompatan tersebut, lalu mengangkat keduanya secara bersamaan hingga hampir bertemu, dan parent merekalah yang akan menjadi LCA dari keduanya.

2.5 Scraping

Web scraping merupakan teknik yang digunakan untuk mengambil data dari website secara otomatis lalu mengubahnya ke format yang dibutuhkan. Cara kerja dari web scraping pada dasarnya mengikuti beberapa langkah utama. Dimulai dari program scraper diberikan URL (alamat website). Scraper mengirimkan request ke website tersebut dan mengambil seluruh konten yang ada, biasanya dalam bentuk HTML. Pada tahap berikutnya, scraper akan membaca struktur HTML tersebut dan mengekstrak data yang diinginkan.

2.6 Pengembangan Website

Pengembangan website merupakan proses pembuatan aplikasi berbasis web yang terdiri dari dua bagian utama, yaitu frontend dan juga backend. Frontend merupakan bagian yang berinteraksi dengan pengguna. Terdapat banyak framework yang dapat digunakan untuk membangun suatu frontend, salah satunya [Next.js](https://nextjs.org/), suatu framework yang berbasis React yang mendukung server-side rendering. Di lain sisi, terdapat juga backend, pengelola dari logika aplikasi, database, dan komunikasi dengan frontend. Pada umumnya bahasa pemrograman memiliki framework-framework backend yang dapat memudahkan pengembangan backend, misalnya saja pada bahasa Golang terdapat Gin, suatu framework yang *light-weight* yang cepat dan juga efisien untuk membangun REST API.

2.7 Bahasa Pemrograman

- Golang

Golang merupakan bahasa pemrograman yang dikembangkan oleh Google dengan berfokus pada performa yang tinggi dan efisien, serta sederhana dan mudah dipahami. Golang cocok digunakan untuk membangun server, API, dan aplikasi yang membutuhkan kecepatan serta skalabilitas tinggi. Selain itu, Golang juga memiliki fitur bawaan untuk concurrency melalui goroutine dan channel. Dalam tugas ini, Golang dipilih karena keunggulan-keunggulannya tadi dan juga tersedia framework Gin yang dapat mempermudah pembangunan website dengan REST API.

- Typescript

Typescript merupakan bahasa pemrograman yang dikembangkan oleh Microsoft sebagai pengembangan dari JavaScript. Perbedaan utamanya ialah adanya static typing, yaitu kemampuan untuk mendefinisikan tipe data pada variabel sehingga membantu mencegah kesalahan sejak tahap penulisan kode. Meskipun demikian, TypeScript tetap kompatibel dengan JavaScript, artinya kode TypeScript dapat dikompilasi menjadi JavaScript sebelum dijalankan di browser atau server.

2.8 Multithreading

Multithreading merupakan konsep menjalankan beberapa thread secara bersamaan dalam satu program untuk meningkatkan performa. Konsep thread mirip dengan Goroutine yang dimiliki oleh Golang, meskipun sebenarnya berbeda. Sebuah *native thread* dapat berisikan sangat banyak goroutine. Goroutine sangat ringan dan sifatnya *asynchronous* menjadikannya tidak saling tunggu dengan goroutine lain. Akan tetapi, diperlukan juga cara agar goroutines yang digunakan dapat berkomunikasi dan menyinkronkan eksekusi mereka. Hal ini dapat diperoleh melalui adanya konsep Channel di Golang. Channel merupakan suatu *transfer pipe* di mana data dapat dituliskan ke dalamnya atau dibaca dari padanya. Satu goroutine dapat mengirimkan data dan goroutine lainnya dapat menerima data dari channel yang sama. Konsep ini dapat diterapkan ketika melakukan traversal pada suatu struktur data tree dan membuat performa menjadi lebih optimal.

BAB III

Aplikasi BFS dan DFS

Berikut ini struktur data tree yang digunakan dalam membentuk DOM tree

```
type Attribute struct {
    Name string
    Value string
}

type Element struct {
    Data string
    IsText bool
    Attributes []Attribute
}

type Node struct {
    Elmt Element
    Children []*Node
    Parent *Node
}
```

3.1. BFS

Berikut ini langkah umum BFS yang dilakukan dalam menelusuri DOM tree.

1. Traversal dimulai dengan memasukkan *root node* ke dalam sebuah antrean (queue) Antrean ini berfungsi untuk mencatat node mana yang harus dikunjungi berikutnya.
2. Selama antrian tidak kosong, algoritma akan mengambil node paling depan dari queue dan memproses node tersebut dengan mencocokkan CSS selector. Setelah itu, semua child dari node yang baru diproses akan dimasukkan ke bagian belakang queue
3. Dikarenakan children dari suatu node dimasukkan ke bagian belakang queue, penelusuran dipaksa untuk menyelesaikan semua elemen yang berada di tingkat kedalaman yang sama terlebih dahulu sebelum lanjut ke level berikutnya
4. Proses berhenti ketika queue telah kosong

Berikut ini mekanisme pemrosesan BFS yang dilakukan dengan memanfaatkan concurrency.

1. Algoritma mengisolasi satu level kedalaman

2. Setiap node pada level terkait akan diproses secara bersamaan menggunakan Goroutines. Dalam proses yang dilakukan, digunakan pembatasan goroutines dengan memanfaatkan semaphore agar tidak membebani memori.
3. Terdapat penggunaan `sync.WaitGroup` yang membuat penelusuran akan menunggu hingga seluruh node di level saat ini selesai diperiksa sebelum mengumpulkan masing-masing anak-anaknya.
4. Meskipun pemrosesan dilakukan secara paralel, hasil akan dimasukkan ke dalam suatu buffer berdasarkan indeks aslinya sehingga urutan elemen yang ditemukan tetap konsisten

3.2. DFS

Berikut ini langkah umum DFS yang dilakukan dalam menelusuri DOM tree.

1. Traversal dimulai dari root node
2. Kunjungi node saat ini dan memproses node tersebut, dalam hal ini dengan mencocokkannya dengan CSS selector
3. Periksa apakah node memiliki child dan jika ditemukan, penelusuran ke dalam child pertama node tersebut akan dilakukan terlebih dahulu secara rekursif daripada memeriksa tetangganya terlebih dahulu
4. Proses ini berlanjut terus hingga mencapai node terdalam dalam satu cabang (leaf node)
5. Setelah satu cabang dijelajahi sepenuhnya, algoritma akan melakukan backtrack ke node parent untuk memastikan apakah masih ada cabang lain yang belum dikunjungi
6. Dari posisi parent, penelusuran akan dilanjutkan ke anak berikutnya sesuai dengan urutan dalam Children dan mengulangi proses yang sama
7. Penelusuran akan berhenti ketika setiap node telah dikunjungi

Berikut ini mekanisme pemrosesan DFS yang dilakukan dengan memanfaatkan goroutines.

1. Penelusuran dimulai dengan memeriksa node saat ini apakah cocok dengan kriteria selector atau tidak dan jika cocok akan dimasukkan ke list hasil lokal.
2. Saat menemukan children, algoritma akan melakukan pengecekan terlebih dahulu apakah ada slot untuk pemrosesan children tersebut atau tidak dan jika ada, goroutine baru akan dihasilkan untuk memproses cabang tersebut. Hal ini memungkinkan cabang, misalnya saja sisi kiri pohon dan sisi kanan

pohon, diproses secara bersamaan. Akan tetapi, jika slot penuh, pemrosesan akan memanfaatkan rekursif biasa di dalam thread yang sama.

3. Penelusuran akan dilakukan sampai menemui leaf.
4. Setelah setiap cabang selesai ditelusuri, cabang tersebut akan melepaskan slot yang digunakan agar bisa dipakai cabang lain. Penggunaan `sync.WaitGroup` memastikan semua goroutine sudah selesai melakukan pemrosesan sebelum menggabungkan setiap hasil.
5. Meskipun diproses secara paralel, hasil dari setiap child disimpan dalam buffer berdasarkan indeks aslinya. Hal ini menjamin keterurutan elemen terjaga.

3.3. LCA

Berikut ini struktur data yang digunakan untuk menyimpan hasil preprocessing

```
type LCASolver struct {  
    nodes          []*Node  
    idByNode map    []*Node]int  
    nodeByID map    [string]int  
    depth          []int  
    ancestor       [][]int  
    logN           int  
}
```

Dalam menyelesaikan permasalahan LCA, perlu dilakukan terlebih dahulu traversal terhadap tree untuk mencatat hubungan orang tua dan menghitung kedalaman masing-masing node.

1. Inisialisasi dasar yang ditandai dengan mengisi baris pertama matriks ancestor dengan orang tua langsung dari setiap node
2. Membangun Binary Lifting dengan menggunakan prinsip bahwa $\text{Ancestor}[k][v] = \text{ancestor}[k-1][\text{ancestor}[k-1][v]]$, di mana k merupakan pangkat dua dari lompatan dan v merupakan indeks node yang sedang diproses

Setelah preprocessing dilakukan, pencarian LCA akan dilakukan dengan dua tahap.

1. Penyamaan kedalaman, di mana node yang lebih dalam akan melakukan “lompat” ke atas (menuju root) hingga berada di kedalaman yang sama dengan node lainnya. Lompatan ini dilakukan menggunakan representasi biner dari perbedaan kedalaman keduanya.
2. Setelah berada di kedalaman yang sejajar, jika kedua node belum sama, keduanya akan berusaha untuk melakukan lompatan sejauh mungkin selagi

hasil lompatannya masih berbeda. Ketika lompatan semakin kecil, mereka pada akhirnya akan berada tepat di bawah ancestor yang sama. Hal akhirnya adalah orang tua langsung ($\text{ancestor}[0][u]$), dengan u merupakan indeks posisi sementara dari salah satu node yang sedang berjalan ke atas menuju root pohon, dari posisi terakhir mereka.

BAB IV

Implementasi dan Pengujian

4.1. Implementasi Algoritma BFS

4.1.1 Struktur Data

Tabel 4.1.1.1 Tabel Struktur Data BFS

Struktur Data	Penjelasan
Attribute	Menyimpan atribut elemen HTML, terdiri dari Name dan Value
Element	Struktur data representasi elemen DOM, terdiri dari Data (tag/isi), IsText (penanda text node), dan Attributes (array of Attribute)
Node	Struktur data representasi tree DOM, terdiri dari Elmt, Children (array of pointer ke Node), dan Parent (pointer ke parent)
Queue	Struktur data antrian untuk traversal level-order
bfsLevelResult	Struktur data <i>helper</i> untuk BFS paralel, berisi matchedNode (node yang lolos selector) dan children (daftar child untuk level berikutnya)

4.1.2 Fungsi dan Prosedur

1. Fungsi bfsSearch(selector string) -> []*Node

Fungsi untuk menjalankan pencarian node pada tree DOM menggunakan algoritma BFS (*level by level*).

```
func (node *Node) bfsSearch(selector string) []*Node {  
    var result []*Node  
  
    // node kosong  
    if node == nil {  
        return result  
    }  
  
    queue := []*Node{node} // antrian node  
  
    for len(queue) > 0 {  
        // ambil node paling depan & hapus dari queue
```

```

curr := queue[0]
queue = queue[1:]

if curr == nil {
    continue
}

// insert node ke hasil sesuai filter
if includeNode(curr, selector) {
    result = append(result, curr)
}

if hasChildren(curr) {
    // insert child ke belakang queue (proses level
    // by level)
    for _, child := range curr.Children {
        if child != nil {
            queue = append(queue, child)
        }
    }
}

return result
}

```

2. Fungsi `bfsSearchParallel(selector string, maxProcess int) -> []*Node`

Fungsi untuk menjalankan pencarian node pada tree DOM menggunakan algoritma BFS Parallel (*concurrent*). Setiap node pada level yang sama dapat diproses dengan *goroutine*, dengan batas konkurensi *maxProces*.

```

func (node *Node) bfsSearchParallel(selector string,
maxProcess int) []*Node {
    if maxProcess <= 1 {
        return node.bfsSearch(selector)
    }

    var result []*Node

```

```

if node == nil {
    return result
}

// Semaphore untuk membatasi jumlah goroutine aktif
// (semacam thread)
sem := make(chan struct{}, maxProcess)
currentLevel := []*Node{node}

for len(currentLevel) > 0 {
    // Simpan hasil per level
    levelResults := make([]bfsLevelResult,
len(currentLevel))
    var wg sync.WaitGroup

    for i, curr := range currentLevel {
        if curr == nil {
            continue
        }

        // Kalau true, ada slot kosong (run child di
        // goroutine)
        // Kalau false, semua slot penuh (lanjut ke
        // proses sinkron)
        if !acquireSem(sem) {
            levelResults[i] = bfsCollectNodeData(curr,
selector)
            continue
        }

        wg.Add(1)
        go func(idx int, curNode *Node) {
            defer wg.Done()
            defer releaseSem(sem)
            levelResults[idx] =
bfsCollectNodeData(curNode, selector)
        }(i, curr)
    }
}

```



```

wg.Wait()

nextLevel := make([]*Node, 0)
for _, lr := range levelResults {
    if lr.matchedNode != nil {
        result = append(result, lr.matchedNode)
    }

    // Gabung child
    if len(lr.children) > 0 {
        nextLevel = append(nextLevel,
lr.children...)
    }
}

// lanjut tingkat berikut
currentLevel = nextLevel
}

return result
}

```

3. Fungsi *helper* bfsCollectNodeData(node *Node, selector string) -> bfsLevelResult

Fungsi *helper* untuk mengambil data yang dibutuhkan BFS paralel dari satu node (apakah node cocok selector dan daftar child valid).

```

func bfsCollectNodeData(node *Node, selector string)
bfsLevelResult {
    res := bfsLevelResult{}

    if node == nil {
        return res
    }

    if includeNode(node, selector) {
        res.matchedNode = node
    }
}

```

```
if hasChildren(node) {
    for _, child := range node.Children {
        if child != nil {
            res.children = append(res.children, child)
        }
    }
}

return res
}
```

4.2. Implementasi Algoritma DFS

4.2.1 Struktur Data

Tabel 4.2.1.1 Tabel Struktur Data DFS

Struktur Data	Penjelasan
Attribute	Menyimpan atribut elemen HTML, terdiri dari Name dan Value
Element	Struktur data representasi elemen DOM, terdiri dari Data (tag/isi), IsText (penanda text node), dan Attributes (array of Attribute)
Node	Struktur data representasi tree DOM, terdiri dari Elmt, Children (array of pointer ke Node), dan Parent (pointer ke parent)
result	Array of pointer Node yang menyimpan node-node hasil pencocokan selector
childResult	Struktur sementara array 2D untuk menyimpan hasil traversal tiap subtree pada DFS paralel sebelum digabung

4.2.2 Fungsi dan Prosedur

1. Fungsi dfsSearch(selector string) -> []*Node

Fungsi untuk menjalankan pencarian node menggunakan algoritma DFS rekursif dalam urutan *pre-order*.

```
func (node *Node) dfsSearch(selector string) []*Node {
    var result []*Node
```

```

// empty node
if node == nil {
    return result
}

// insert node ke hasil sesuai filter
if includeNode(node, selector) {
    result = append(result, node)
}

if hasChildren(node) {
    // explore hingga paling dalam, baru pindah ke
    // cabang selanjutnya
    for _, child := range node.Children {
        if child != nil {
            childResult := child.dfsSearch(selector)
            result = append(result, childResult...)
        }
    }
}

return result
}

```

2. Fungsi `dfsSearchParallel(selector string, maxProcess int) -> []*Node`

Fungsi untuk menjalankan DFS secara paralel. Jika *maxProcess* > 1, traversal subtree dapat dijalankan *concurrent* dengan batas semaphore.

```

func (node *Node) dfsSearchParallel(selector string,
maxProcess int) []*Node {
    if maxProcess <= 1 {
        return node.dfsSearch(selector)
    }

    // Semaphore untuk membatasi jumlah goroutine aktif
    // (semacam thread)
    sem := make(chan struct{}, maxProcess)
    return dfsSearchConcurrent(node, selector, sem)
}

```

```
}
```

3. Fungsi `dfsSearchConcurrent(node *Node, selector string, sem chan struct{}) -> []*Node`

Fungsi inti DFS paralel untuk memproses node saat ini dan seluruh subtree child secara *concurrent* dengan tetap menjaga urutan penggabungan hasil sesuai urutan *children*.

```
func dfsSearchConcurrent(node *Node, selector string, sem
chan struct{}) []*Node {
    var result []*Node

    if node == nil {
        return result
    }

    if includeNode(node, selector) {
        result = append(result, node)
    }

    if !hasChildren(node) {
        return result
    }

    childResults := make([]*Node, len(node.Children))
    var wg sync.WaitGroup

    for i, child := range node.Children {
        if child == nil {
            continue
        }

        // Kalau true, ada slot kosong (run child di
        goroutine)
        // Kalau false, semua slot penuh (lanjut ke proses
        sinkron)
        if !acquireSem(sem) {
            childResults[i] = dfsSearchConcurrent(child,
            selector, sem)
```

```

        continue
    }

    wg.Add(1)
    go func(idx int, childNode *Node) {
        defer wg.Done()

        // Semaphore dilepaskan supaya slot process bisa
        // dipakai subtree lain
        defer releaseSem(sem)
        childResults[idx] =
dfsSearchConcurrent(childNode, selector, sem)
    }(i, child)
}

wg.Wait()

// Gabungkan child
for _, childResult := range childResults {
    result = append(result, childResult...)
}

return result
}

```

4. Fungsi `acquireSem(sem chan struct{}) -> bool`

Fungsi untuk mencoba mengambil satu slot semaphore tanpa blocking. Mengembalikan true jika berhasil, false jika channel penuh.

```

func acquireSem(sem chan struct{}) bool {
    // Kalau channel buffer penuh, langsung return false
    // (tidak menunggu)
    select {
    case sem <- struct{}{}:
        return true
    default:
        return false
    }
}

```

5. Fungsi `releaseSem(sem chan struct{})`

Fungsi untuk melepas satu slot semaphore setelah proses subtree selesai.

```
func releaseSem(sem chan struct{}) {  
    // release 1 semaphore yang sebelumnya diambil oleh  
    acquireSem  
    <-sem  
}
```

4.3. Implementasi Algoritma Binary Lifting

4.3.1 Struktur Data

Tabel 4.3.1.1 Tabel Struktur Data Binary Lifting

Struktur Data	Penjelasan
Attribute	Menyimpan atribut elemen HTML, terdiri dari Name dan Value
Element	Struktur data representasi elemen DOM, terdiri dari Data (tag/isi), IsText (penanda text node), dan Attributes (array of Attribute)
Node	Struktur data representasi tree DOM, terdiri dari Elmt, Children (array of pointer ke Node), dan Parent (pointer ke parent)
LCASolver	Struktur utama solver LCA berisi nodes, pemetaan id, depth, tabel ancestor, dan logN
nodes	Array node hasil flatten traversal DFS, setiap node punya indeks integer unik.
idByNode	Pemetaan pointer node ke indeks integer internal solver.
nodeByID	Pemetaan ID string ke indeks integer internal solver
depth	Menyimpan kedalaman tiap node dari root
ancestor	Tabel binary lifting, $\text{ancestor}[k][v]$ = leluhur ke- 2^k dari node v. Jika tidak ada bernilai -1
logN	Nilai maksimum k

4.3.2 Fungsi dan Prosedur

1. Fungsi NewLCASolver(root Node pointer, ids []string) -> LCASolver pointer, error

Fungsi *preprocessing* untuk membangun semua struktur yang dibutuhkan binary lifting.

```
func NewLCASolver(root *Node, ids []string) (*LCASolver,
error) {
    if root == nil {
        return nil, fmt.Errorf("root is nil")
    }

    s := &LCASolver{
        idByNode: map[*Node]int{},
        nodeByID: map[string]int{},
    }

    var walk func(n *Node, d int)
    walk = func(n *Node, d int) {
        if n == nil {
            return
        }
        idx := len(s.nodes)
        s.nodes = append(s.nodes, n)
        s.idByNode[n] = idx
        s.depth = append(s.depth, d)
        for _, c := range n.Children {
            walk(c, d+1)
        }
    }
    walk(root, 0)

    n := len(s.nodes)
    if len(ids) != n {
        return nil, fmt.Errorf("id slice length %d != node
count %d", len(ids), n)
    }
    for i, id := range ids {
        s.nodeByID[id] = i
    }
}
```

```

}

logN := 1
for (1 << logN) < n {
    logN++
}
s.logN = logN

s.ancestor = make([][]int, logN+1)
for k := 0; k <= logN; k++ {
    s.ancestor[k] = make([]int, n)
    for i := range s.ancestor[k] {
        s.ancestor[k][i] = -1
    }
}

for i, node := range s.nodes {
    if node.Parent != nil {
        if pi, ok := s.idByNode[node.Parent]; ok {
            s.ancestor[0][i] = pi
        }
    }
}

for k := 1; k <= logN; k++ {
    for v := 0; v < n; v++ {
        mid := s.ancestor[k-1][v]
        if mid == -1 {
            s.ancestor[k][v] = -1
            continue
        }
        s.ancestor[k][v] = s.ancestor[k-1][mid]
    }
}

return s, nil
}

```


2. Fungsi LCAByID(idA, idB string) -> string, error

Fungsi query utama (API). Input node berupa ID string, output berupa ID string dari Lowest Common Ancestor.

```
func (s *LCASolver) LCAByID(idA, idB string) (string, error) {
    a, okA := s.nodeByID[idA]
    b, okB := s.nodeByID[idB]
    if !okA {
        return "", fmt.Errorf("unknown node id: %s", idA)
    }
    if !okB {
        return "", fmt.Errorf("unknown node id: %s", idB)
    }

    lca := s.lcaIdx(a, b)
    if lca == -1 {
        return "", fmt.Errorf("no common ancestor")
    }

    for id, idx := range s.nodeByID {
        if idx == lca {
            return id, nil
        }
    }
    return "", fmt.Errorf("lca id not found")
}
```

3. Fungsi lcaIdx(u, v int) -> int

Fungsi inti binary lifting untuk mencari LCA dua indeks node.

```
func (s *LCASolver) lcaIdx(u, v int) int {
    if s.depth[u] < s.depth[v] {
        u, v = v, u
    }
    diff := s.depth[u] - s.depth[v]
    for k := 0; k <= s.logN && diff > 0; k++ {
        if diff&1 == 1 {
            u = s.ancestor[k][u]
            if u == -1 {
                return -1
            }
        }
        diff >>= 1
    }
    return u
}
```

```

        return -1
    }
}
diff >>= 1
}
if u == v {
    return u
}
for k := s.logN; k >= 0; k-- {
    au := s.ancestor[k][u]
    av := s.ancestor[k][v]
    if au != av {
        u, v = au, av
        if u == -1 || v == -1 {
            return -1
        }
    }
}
return s.ancestor[0][u]
}

```

4.4. Hasil Pengujian

Web yang di-scrape dalam pengujian adalah <https://youtube.com> dengan limit 5 (hanya untuk tc 1 & 2) dan html yang digunakan (pengecekan semua) adalah seperti berikut

```

<!doctype html>
<html>
<head>
  <title>TC Playground</title>
</head>
<body id="root-body" class="main page">
  <header id="top" class="site-header">
    <nav class="menu">
      <a id="home-link" class="item active">Home</a>
      <a id="docs-link" class="item">Docs</a>
    </nav>
  </header>

  <main id="content" class="container">
    <section id="hero" class="block hero">
      <h1 id="hero-title" class="title">Welcome</h1>
      <p id="hero-desc" class="desc lead">Intro text</p>
    </section>
  </main>
</body>
</html>

```

```

</section>

<section id="articles" class="block list-wrap">
  <article id="a1" class="card featured">
    <h2 id="a1-title" class="title">Article 1</h2>
    <p id="a1-text" class="text">Lorem ipsum</p>
    <ul id="a1-tags" class="tags">
      <li id="a1-tag-1" class="tag item">go</li>
      <li id="a1-tag-2" class="tag item active">algorithms</li>
    </ul>
  </article>

  <article id="a2" class="card">
    <h2 id="a2-title" class="title">Article 2</h2>
    <p id="a2-text" class="text">Dolor sit amet</p>
    <ul id="a2-tags" class="tags">
      <li id="a2-tag-1" class="tag item">web</li>
      <li id="a2-tag-2" class="tag item">nextjs</li>
    </ul>
  </article>
</section>

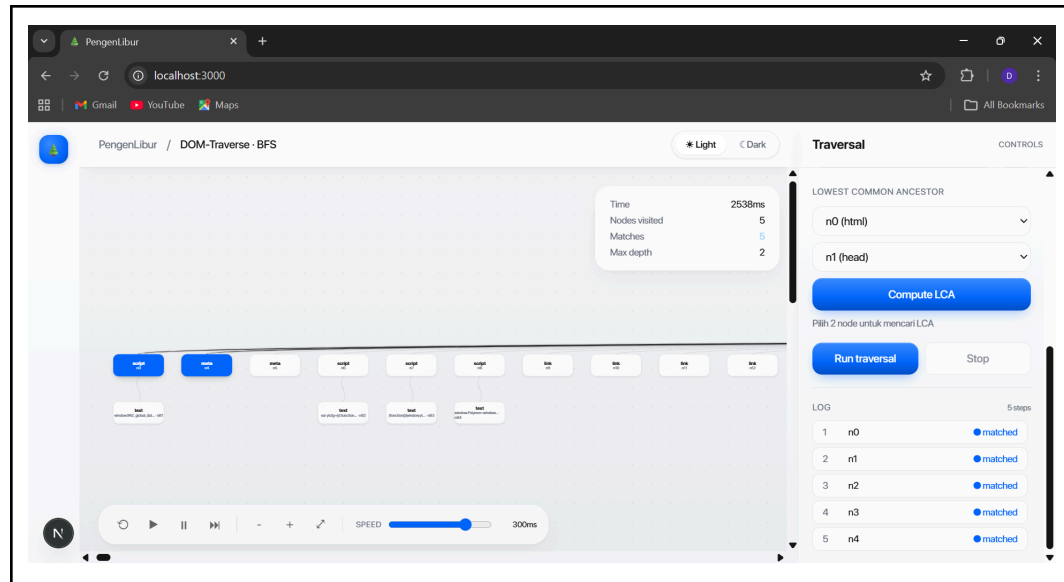
<aside id="sidebar" class="panel side">
  <div id="profile" class="widget profile">
    <h3 id="profile-title" class="title">Profile</h3>
    <p id="profile-bio" class="text">Bio</p>
  </div>
  <div id="quick-links" class="widget links">
    <a id="ql-1" class="item">Guide</a>
    <a id="ql-2" class="item active">API</a>
  </div>
</aside>
</main>

<footer id="footer" class="site-footer">
  <small id="copyright" class="muted">2026</small>
</footer>
</body>
</html>

```

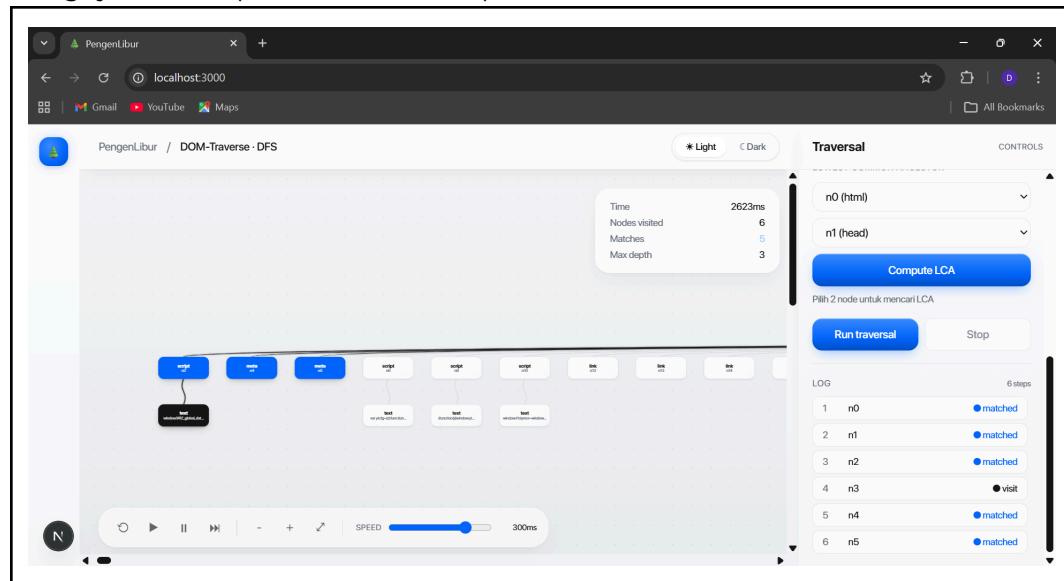
1. Test Case 1

Pengujian BFS * (*Universal Selector*)



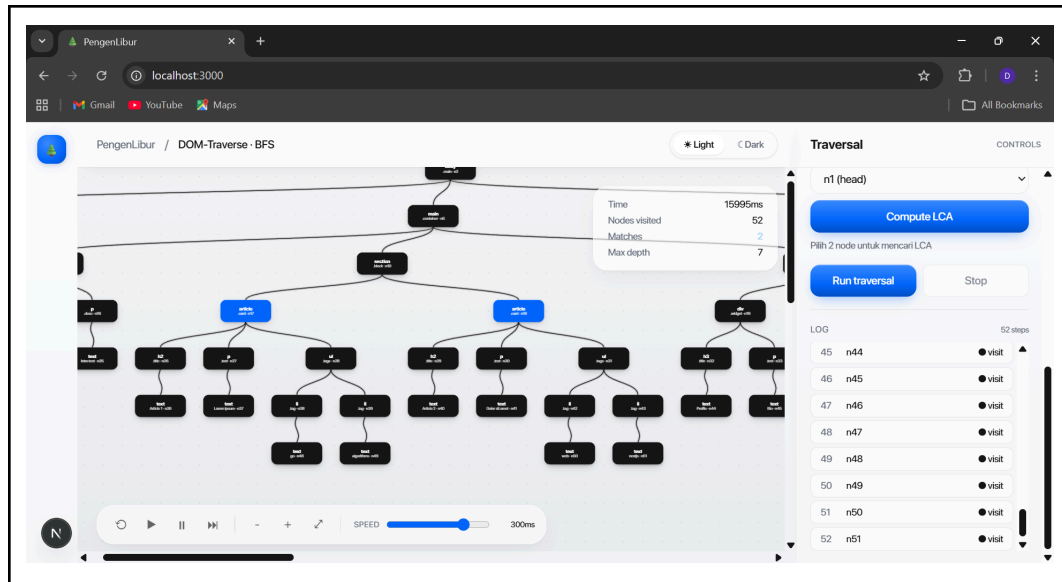
2. Test Case 2

Pengujian DFS * (*Universal Selector*)



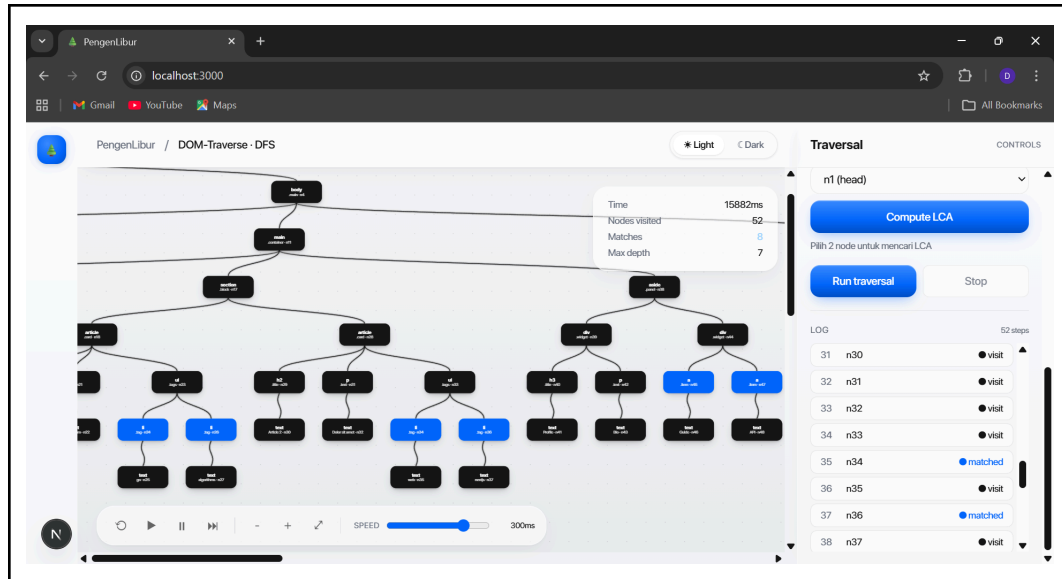
3. Test Case 3

Pengujian BFS tag article (*all & html*)



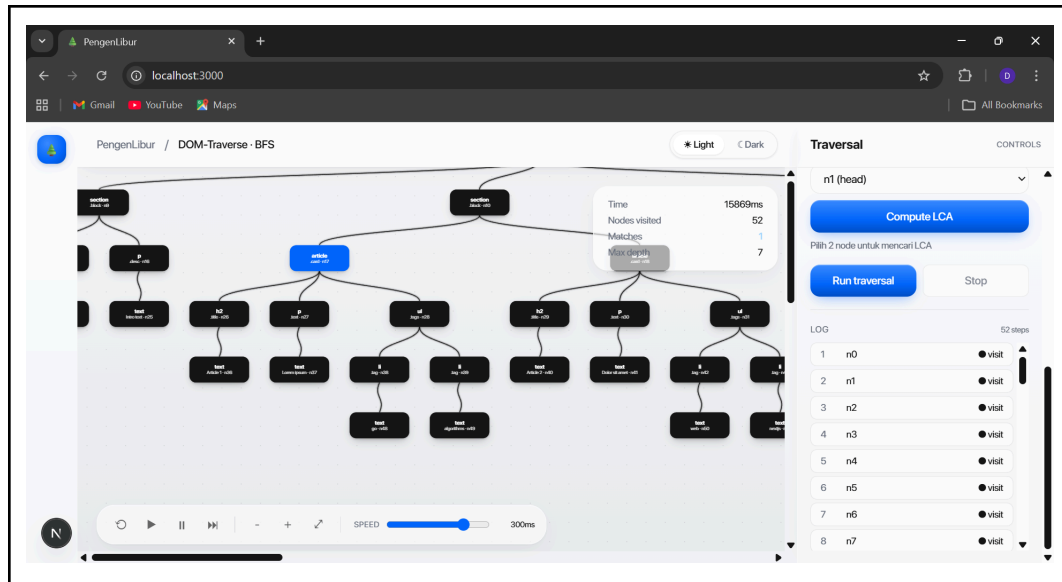
4. Test Case 4

Pengujian DFS class .item (all & html)



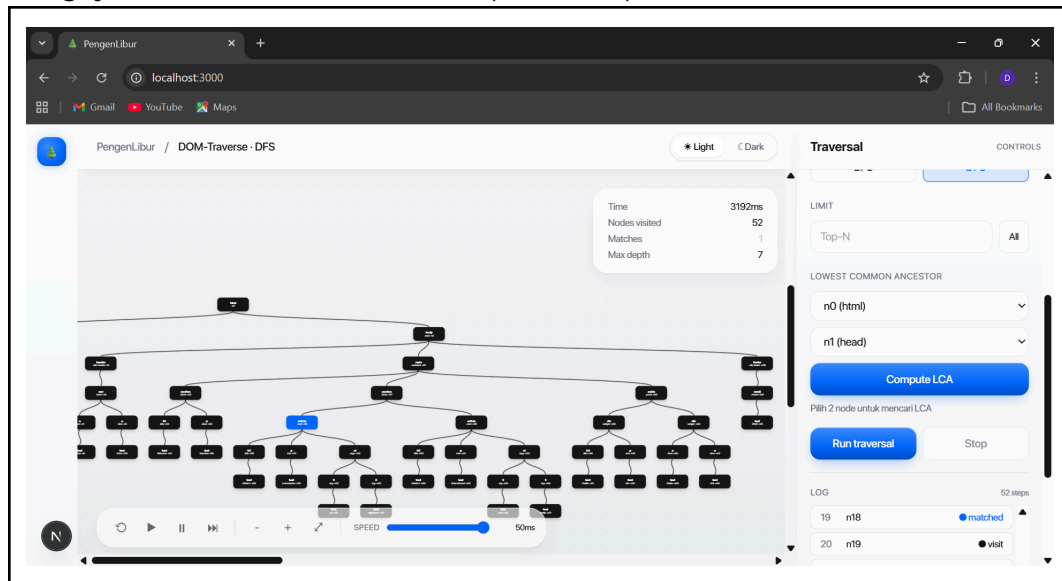
5. Test Case 5

Pengujian BFS ID #a1 (all & html)



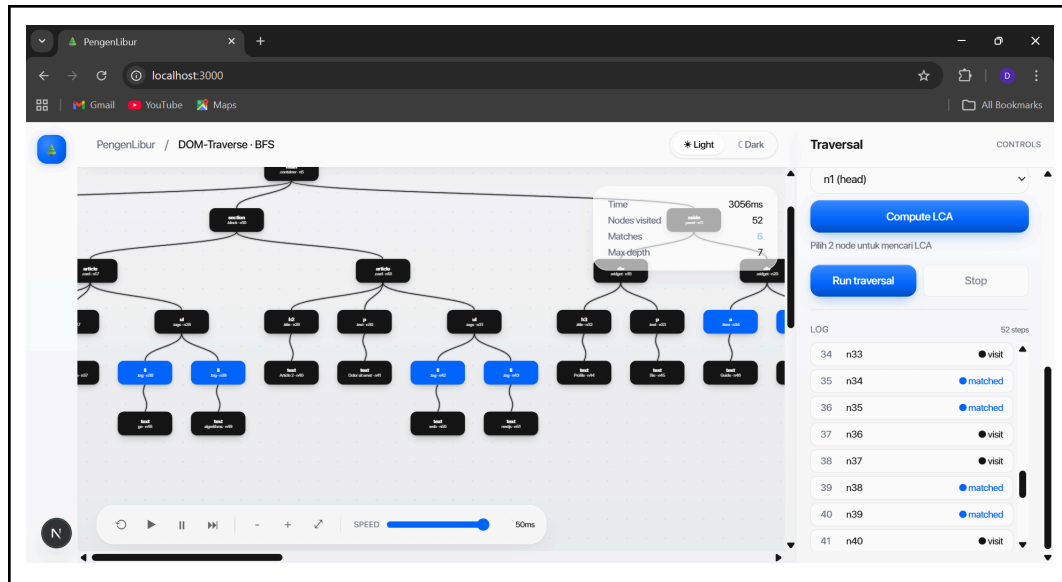
6. Test Case 6

Pengujian DFS article.card.featured (*all & html*)



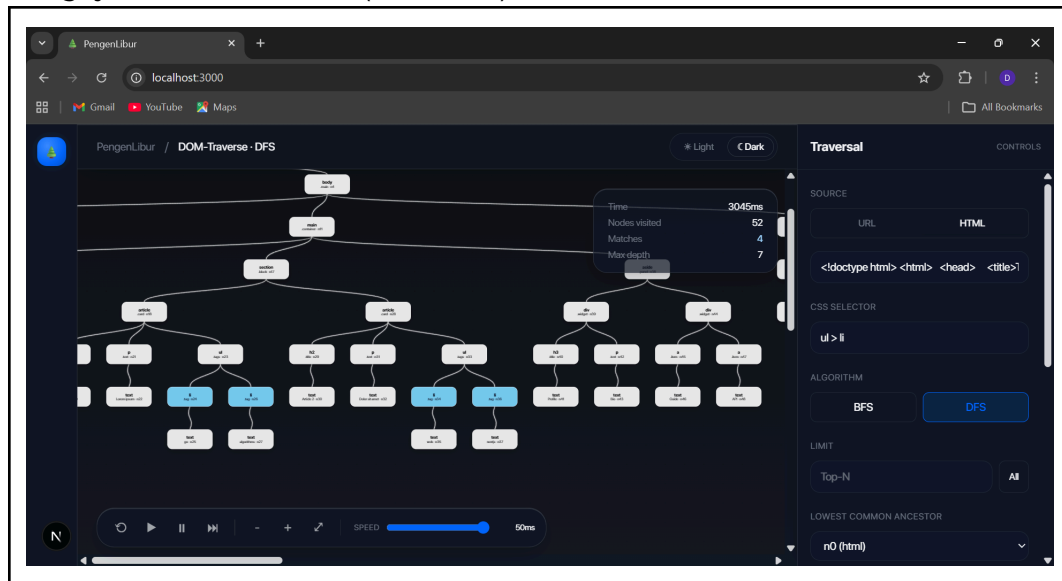
7. Test Case 7

Pengujian BFS main.item (*all & html*)



8. Test Case 8

Pengujian DFS child ul > li (*all & html*)



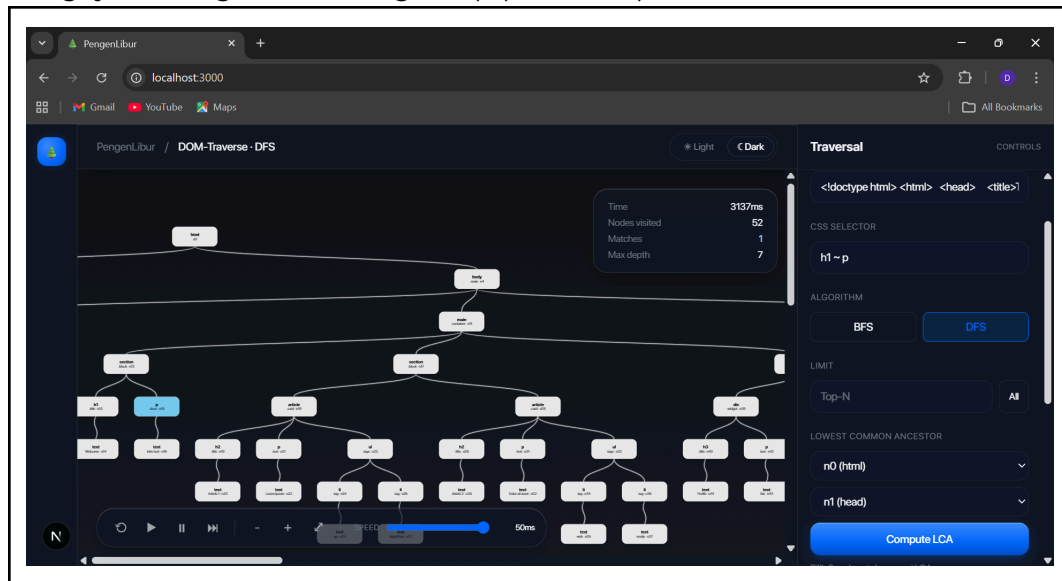
9. Test Case 9

Pengujian BFS adjacent sibling h1 + p (*all & html*)



10. Test Case 10

Pengujian DFS general sibling h1 ~ p (*all & html*)



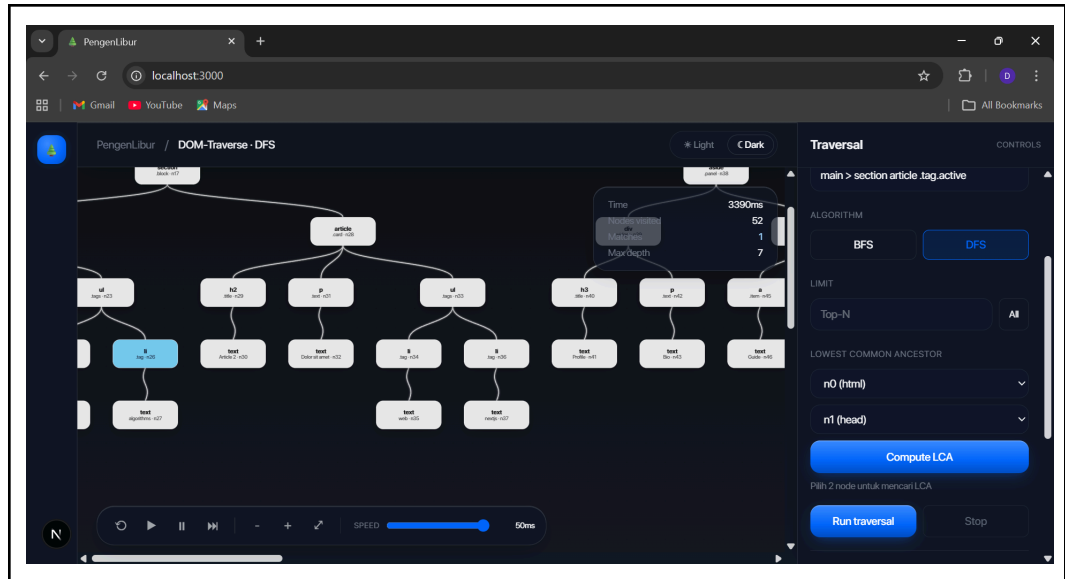
11. Test Case 11

Pengujian BFS (*tidak ada hasil*) section.unkown (*all & html*)



12. Test Case 12

Pengujian DFS main > section article .tag.active (all & html)



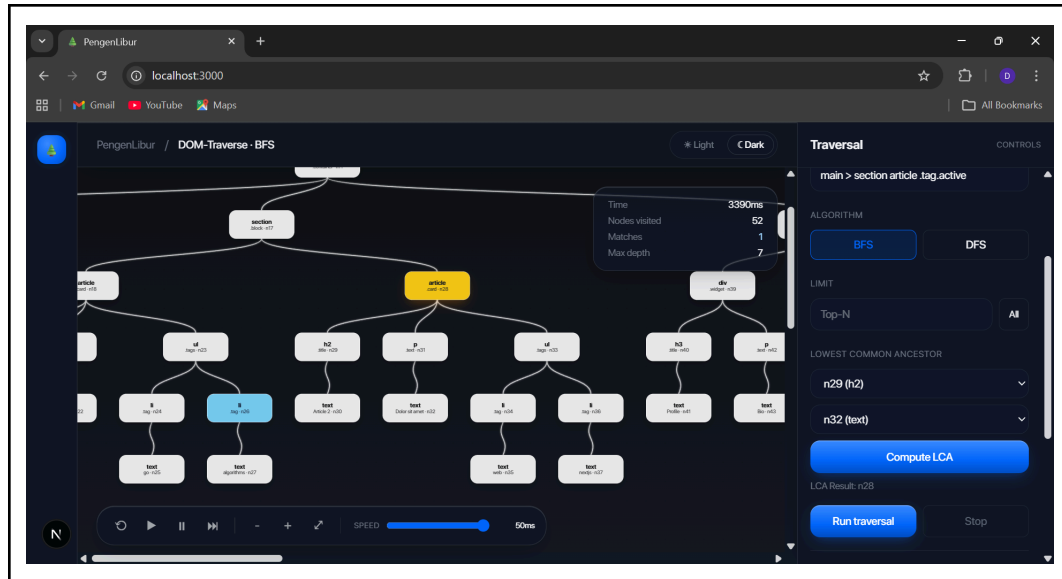
13. Test Case 13

Pengujian LCA section (n17) dengan text (n22)



14. Test Case 14

Pengujian LCA h2 (n29) dengan text (n32)



BAB V

Kesimpulan dan Saran

5.1 Kesimpulan

Melalui Tugas Besar II Strategi Algoritma, kami belajar cara lebih dalam mengimplementasikan algoritma BFS dan DFS untuk menelusuri suatu tree, khususnya DOM tree yang dibangun dengan melakukan scraping HTML dan melakukan parsing untuk membangun tree-nya. Selain itu juga, kami jadi belajar mengenai pengembangan web menggunakan Golang dengan Framework Gin dan Typescript dalam framework Next.js. Selain itu dari pengujian yang telah kami lakukan, baik DFS maupun BFS dapat mencapai suatu node terkadang bisa lebih cepat dibandingkan yang lainnya bergantung pada input yang diberikan.

5.2 Saran

Berikut adalah saran dan refleksi dari kelompok untuk kelompok di masa depan.

- Komunikasi dengan lebih aktif dan terbuka terhadap keadaan yang dihadapi masing-masing anggota
- Memperbaiki penulisan komentar dan dokumentasi agar anggota kelompok lain lebih mudah memahami kode yang telah dibuat

5.3 Refleksi

Pengerjaan Tugas Besar 2 Strategi Algoritma memberikan pengetahuan dan juga pengalaman mengenai algoritma penelusuran dalam graf, seperti BFS dan juga DFS. Selain itu, kami juga jadi belajar terkait Least Common Ancestor. Kami juga belajar mengenai parsing HTML dan membentuknya jadi DOM Tree. Selain itu, kami juga belajar web scraping sederhana untuk web yang sifatnya statis. Kami juga menyadari bahwa komunikasi merupakan hal yang penting dalam pengerjaan tugas ini.

Lampiran

Pernyataan tidak melakukan kecurangan yang ditandatangani

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Made Branenda Jordhy



Irvin Tandiarrang Sumual



Valentino Daniel Kusumo

Tautan repository GitHub

Tautan GitHub FE (*Frontend*): https://github.com/ethj0r/Tubes2_PengenLibur_FE

Tautan Github BE (*Backend*): https://github.com/ethj0r/Tubes2_PengenLibur_BE

Link Web (*Deploy*) – masih dinonaktifkan agar kredit tidak habis:

<http://domtree.eastasia.cloudapp.azure.com>

No	Poin	Ya	Tidak
1	Aplikasi berhasil di kompilasi tanpa kesalahan	✓	
2	Aplikasi berhasil dijalankan	✓	
3	Aplikasi dapat menerima input URL web, pilihan algoritma, CSS selector, dan jumlah hasil	✓	
4	Aplikasi dapat melakukan scraping terhadap web pada input	✓	
5	Aplikasi dapat menampilkan visualisasi pohon DOM	✓	
6	Aplikasi dapat menelusuri pohon DOM dan menampilkan hasil penelusuran	✓	
7	Aplikasi dapat menandai jalur tempuh oleh algoritma	✓	
8	Aplikasi dapat menyimpan jalur yang ditempuh algoritma dalam traversal log	✓	
9	[Bonus] Membuat video		✓
10	[Bonus] Deploy aplikasi	✓	
11	[Bonus] Implementasi animasi pada penelusuran pohon	✓	
12	[Bonus] Implementasi multithreading	✓	
13	[Bonus] Implementasi LCA Binary Lifting	✓	

Daftar Pustaka

- [1] Donovan, A. A., & Kernighan, B. W. (2015). *The Go programming language*. Addison-Wesley.
- [2] GeeksforGeeks. (n.d.). *Introduction to tree data structure*.
- [3] Gin-Gonic. (n.d.). *Gin Web Framework*. GitHub. <https://github.com/gin-gonic/gin>
- [4] Kristanto, J. (n.d.). *Understanding DFS and BFS: Key concepts for tree traversal*. Medium.
- [5] Koç, B. (n.d.). *Binary lifting: Turbocharging tree traversal and LCA queries*. Medium.
- [6] Microsoft. (n.d.). *TypeScript: JavaScript with syntax for types*.
- [7] ParseHub. (n.d.). *What is web scraping?* Diakses dari <https://www.parsehub.com/blog/what-is-web-scraping/>
- [8] Revou. (n.d.). *Web scraping*. Diakses dari <https://www.revou.co/kosakata/web-scraping>
- [9] Yadav, A. (n.d.). *Goroutines and channels in Go: A beginner's guide*. Medium.